

4.1 Y86-64 指令集体系结构

定义一个指令集体系结构(例如 Y86-64)包括定义各种状态单元、指令集和它们的编码、一组编程规范和异常事件处理。

4.1.1 程序员可见的状态

如图 4-1 所示, Y86-64 程序中的每条指令都会读取或修改处理器状态的某些部分。这称为程序员可见状态, 这里的“程序员”既可以是用汇编代码写程序的人, 也可以是产生机器级代码的编译器。在处理器实现中, 只要我们保证机器级程序能够访问程序员可见状态, 就不需要完全按照 ISA 暗示的方式来表示和组织这个处理器状态。Y86-64 的状态类似于 x86-64。有 15 个程序寄存器: %rax、%rcx、%rdx、%rbx、%rsp、%rbp、%rsi、%rdi 和 %r8 到 %r14。(我们省略了 x86-64 的寄存器 %r15 以简化指令的编码。)每个程序寄存器存储一个 64 位的字。寄存器 %rsp 被入栈、出栈、调用和返回指令作为栈指针。除此之外, 寄存器没有固定的含义或固定值。有 3 个一位的条件码: ZF、SF 和 OF, 它们保存着最近的算术或逻辑指令所造成影响的有关信息。程序计数器(PC)存放当前正在执行指令的地址。

内存从概念上来说就是一个很大的字节数组, 保存着程序和数据。Y86-64 程序用虚拟地址来引用内存位置。硬件和操作系统软件联合起来将虚拟地址翻译成实际或物理地址, 指明数据实际存在内存中哪个地方。第 9 章将更详细地研究虚拟内存。现在, 我们认为虚拟内存系统向 Y86-64 程序提供了一个单一的字节数组映像。

程序状态的最后一个部分是状态码 Stat, 它表明程序执行的总体状态。它会指示是正常运行, 还是出现了某种异常, 例如当一条指令试图去读非法的内存地址时。在 4.1.4 节中会讲述可能的状态码以及异常处理。

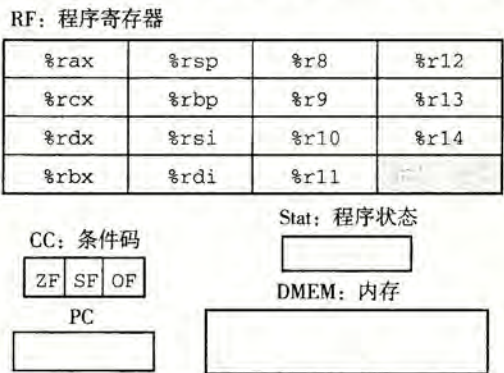


图 4-1 Y86-64 程序员可见状态。同 x86-64 一样, Y86-64 的程序可以访问和修改程序寄存器、条件码、程序计数器(PC)和内存。状态码指明程序是否运行正常, 或者发生了某个特殊事件

4.1.2 Y86-64 指令

图 4-2 给出了 Y86-64 ISA 中各个指令的简单描述。这个指令集就是我们处理器实现的目标。Y86-64 指令集基本上是 x86-64 指令集的一个子集。它只包括 8 字节整数操作, 寻址方式较少, 操作也较少。因为我们只有 8 字节数据, 所以称之为“字(word)”不会有任何歧义。在这个图中, 左边是指令的汇编码表示, 右边是字节编码。图 4-3 给出了一些指令更详细的内容。汇编代码格式类似于 x86-64 的 ATT 格式。

下面是 Y86-64 指令的一些细节。

- x86-64 的 movq 指令分成了 4 个不同的指令: irmovq、rrmovq、rmmovq 和 rrmovq, 分别显式地指明源和目的格式。源可以是立即数(i)、寄存器(r)或内存(m)。指令名字的第一个字母就表明了源的类型。目的可以是寄存器(r)或内存(m)。指令名字的第二个字母指明了目的的类型。在决定如何实现数据传送时, 显式地指明数据传送的

这 4 种类型是很有帮助的。

两个内存传送指令中的内存引用方式是简单的基址和偏移量形式。在地址计算中，我们不支持第二变址寄存器(second index register)和任何寄存器值的伸缩(scaling)。

同 x86-64 一样，我们不允许从一个内存地址直接传送到另一个内存地址。另外，也不允许将立即数传送到内存。

- 有 4 个整数操作指令，如图 4-2 中的 OPq。它们是 addq、subq、andq 和 xorq。它们只对寄存器数据进行操作，而 x86-64 还允许对内存数据进行这些操作。这些指令会设置 3 个条件码 ZF、SF 和 OF(零、符号和溢出)。
- 7 个跳转指令(图 4-2 中的 jXX)是 jmp、jle、jl、je、jne、jge 和 jg。根据分支指令的类型和条件代码的设置来选择分支。分支条件和 x86-64 的一样(见图 3-15)。
- 有 6 个条件传送指令(图 4-2 中的 cmovXX)：cmovle、cmovl、cmove、cmovne、cmovge 和 cmovg。这些指令的格式与寄存器-寄存器传送指令 rrmovq 一样，但是只有当条件码满足所需要的约束时，才会更新目的寄存器的值。
- call 指令将返回地址入栈，然后跳到目的地址。ret 指令从这样的调用中返回。
- pushq 和 popq 指令实现了入栈和出栈，就像在 x86-64 中一样。
- halt 指令停止指令的执行。x86-64 中有一个与之相当的指令 hlt。x86-64 的应用程序不允许使用这条指令，因为它会导致整个系统暂停运行。对于 Y86-64 来说，执行 halt 指令会导致处理器停止，并将状态码设置为 HLT(参见 4.1.4 节)。

字节	0	1	2	3	4	5	6	7	8	9
halt	0	0								
nop	1	0								
rrmovq rA, rB	2	0	rA	rB						
irmovq V, rB	3	0	F	rB					V	
rmmovq rA, D(rB)	4	0	rA	rB					D	
rrmovq D(rB), rA	5	0	rA	rB					D	
OPq rA, rB	6	fn	rA	rB						
jXX Dest	7	fn							Dest	
cmovXX rA, rB	2	fn	rA	rB						
call Dest	8	0							Dest	
ret	9	0								
pushq rA	A	0	rA	F						
popq rA	B	0	rA	F						

图 4-2 Y86-64 指令集。指令编码长度从 1 个字节到 10 个字节不等。一条指令含有一个单字节的指令指示符，可能含有一个单字节的寄存器指示符，还可能含有一个 8 字节的常数字。字段 fn 指明是某个整数操作(OPq)、数据传送条件(cmovXX)或是分支条件(jXX)。所有的数值都用十六进制表示

4.1.3 指令编码

图 4-2 还给出了指令的字节级编码。每条指令需要 1~10 个字节不等，这取决于需要哪些字段。每条指令的第一个字节表明指令的类型。这个字节分为两个部分，每部分 4

位：高 4 位是代码(code)部分，低 4 位是功能(function)部分。如图 4-2 所示，代码值为 0~0xB。功能值只有在在一组相关指令共用一个代码时才有用。图 4-3 给出了整数操作、分支和条件传送指令的具体编码。可以观察到，rrmovq 与条件传送有同样的指令代码。可以把它看作是一个“无条件传送”，就好像 jmp 指令是无条件跳转一样，它们的功能代码都是 0。



图 4-3 Y86-64 指令集的功能码。这些代码指明是某个整数操作、分支条件还是数据传送条件。这些指令是图 4-2 中所示的 OPq、jXX 和 cmovXX

如图 4-4 所示，15 个程序寄存器中每个都有一个相对应的范围在 0 到 0xE 之间的寄存器标识符(register ID)。Y86-64 中的寄存器编号跟 x86-64 中的相同。程序寄存器存在 CPU 中的一个寄存器文件中，这个寄存器文件就是一个小的、以寄存器 ID 作为地址的随机访问存储器。在指令编码中以及在我们的硬件设计中，当需要指明不应访问任何寄存器时，就用 ID 值 0xF 来表示。

数字	寄存器名字	数字	寄存器名字
0	%rax	8	%r8
1	%rcx	9	%r9
2	%rdx	A	%r10
3	%rbx	B	%r11
4	%rsp	C	%r12
5	%rbp	D	%r13
6	%rsi	E	%r14
7	%rdi	F	无寄存器

图 4-4 Y86-64 程序寄存器标识符。15 个程序寄存器中每个都有一个相对应的标识符(ID)，范围为 0~0xE。如果指令中某个寄存器字段的 ID 值为 0xF，就表明此处没有寄存器操作数

有的指令只有一个字节长，而有的需要操作数的指令编码就更长一些。首先，可能有附加的寄存器指示符字节(register specifier byte)，指定一个或两个寄存器。在图 4-2 中，这些寄存器字段称为 rA 和 rB。从指令的汇编代码表示中可以看到，根据指令类型，指令可以指定用于数据源和目的的寄存器，或是用于地址计算的基址寄存器。没有寄存器操作数的指令，例如分支指令和 call 指令，就没有寄存器指示符字节。那些只需要一个寄存器操作数的指令(irmovq、pushq 和 popq)将另一个寄存器指示符设为 0xF。这种约定在我们的处理器实现中非常有用。

有些指令需要一个附加的 4 字节常数字(constant word)。这个字能作为 irmovq 的立即数数据，rrmovq 和 rrmovq 的地址指示符的偏移量，以及分支指令和调用指令的目的地址。注意，分支指令和调用指令的目的一个绝对地址，而不像 IA32 中那样使用 PC

(程序计数器)相对寻址方式。处理器使用 PC 相对寻址方式,分支指令的编码会更简洁,同时这样也能允许代码从内存的一部分复制到另一部分而不需要更新所有的分支目标地址。因为我们更关心描述的简单性,所以就使用了绝对寻址方式。同 IA32 一样,所有整数采用小端法编码。当指令按照反汇编格式书写时,这些字节就以相反的顺序出现。

例如,用十六进制来表示指令 `rmmovq %rsp, 0x123456789abcd(%rdx)` 的字节编码。从图 4-2 我们可以看到, `rmmovq` 的第一个字节为 40。源寄存器 `%rsp` 应该编码放在 rA 字段中,而基址寄存器 `%rdx` 应该编码放在 rB 字段中。根据图 4-4 中的寄存器编号,我们得到寄存器指示符字节 42。最后,偏移量编码放在 8 字节的常数字中。首先在 `0x123456789abcd` 的前面填充上 0 变成 8 个字节,变成字节序列 `00 01 23 45 67 89 ab cd`。写成按字节反序就是 `cd ab 89 67 45 23 01 00`。将它们都连接起来就得到指令的编码 `4042cdab896745230100`。

指令集的一个重要性质就是字节编码必须有唯一的解释。任意一个字节序列要么是一个唯一的指令序列的编码,要么就不是一个合法的字节序列。Y86-64 就具有这个性质,因为每条指令的第一个字节有唯一的代码和功能组合,给定这个字节,我们就可以决定所有其他附加字节的长度和含义。这个性质保证了处理器可以无二义性地执行目标代码程序。即使代码嵌入在程序的其他字节中,只要从序列的第一个字节开始处理,我们仍然可以很容易地确定指令序列。反过来说,如果不知道一段代码序列的起始位置,我们就不能准确地确定怎样将序列划分成单独的指令。对于试图直接从目标代码字节序列中抽取出机器级程序的反汇编程序和其他一些工具来说,这就带来了问题。



练习题 4.1 确定下面的 Y86-64 指令序列的字节编码。“`.pos 0x100`”那一行表明这段目标代码的起始地址应该是 `0x100`。

```
.pos 0x100 # Start code at address 0x100
    irmovq $15,%rbx
    rrmovq %rbx,%rcx
loop:
    rmmovq %rcx,-3(%rbx)
    addq   %rbx,%rcx
    jmp   loop
```



练习题 4.2 确定下列每个字节序列所编码的 Y86-64 指令序列。如果序列中有不合法的字节,指出指令序列中不合法值出现的位置。每个序列都先给出了起始地址,冒号,然后是字节序列。

- A. `0x100: 30f3fcffffffffffffffff40630008000000000000`
- B. `0x200: a06f800c02000000000000000030f30a0000000000000090`
- C. `0x300: 505407000000000000000010f0b01f`
- D. `0x400: 611373000400000000000000`
- E. `0x500: 6362a0f0`

旁注 比较 x86-64 和 Y86-64 的指令编码

同 x86-64 中的指令编码相比, Y86-64 的编码简单得多,但是没那么紧凑。在所有的 Y86-64 指令中,寄存器字段的位置都是固定的,而在不同的 x86-64 指令中,它们的位置是不一样的。x86-64 可以将常数值编码成 1、2、4 或 8 个字节,而 Y86-64 总是将常数值编码成 8 个字节。

旁注 RISC 和 CISC 指令集

x86-64 有时称为“复杂指令集计算机”(CISC, 读作“sisk”), 与“精简指令集计算机”(RISC, 读作“risk”)相对。从历史上看, 先出现了 CISC 机器, 它从最早的计算机演化而来。到 20 世纪 80 年代早期, 随着机器设计者加入了很多新指令来支持高级任务(例如处理循环缓冲区, 执行十进制数计算, 以及求多项式的值), 大型机和小型机的指令集已经变得非常庞大了。最早的微处理器出现在 20 世纪 70 年代早期, 因为当时的集成电路技术极大地制约了一块芯片上能实现些什么, 所以它们的指令集非常有限。微处理器发展得很快, 到 20 世纪 80 年代早期, 大型机和小型机的指令集复杂度一直都在增加。x86 家族沿着这条道路发展到 IA32, 最近是 x86-64。即使是 x86 系列也仍然在不断地变化, 基于新出现的应用的需要, 增加新的指令类。

20 世纪 80 年代早期, RISC 的设计理念是作为上述发展趋势的一种替代而发展起来的。IBM 的一组硬件和编译器专家受到 IBM 研究员 John Cocke 的很大影响, 认为他们可以为更简单的指令集形式产生高效的代码。实际上, 许多加到指令集中的高级指令很难被编译器产生, 所以也很少被用到。一个较为简单的指令集可以用很少的硬件实现, 能以高效的流水线结构组织起来, 类似于本章后面描述的情况。直到多年以后 IBM 才将这个理念商品化, 开发出了 Power 和 PowerPC ISA。

加州大学伯克利分校的 David Patterson 和斯坦福大学的 John Hennessy 进一步发展了 RISC 的概念。Patterson 将这种新的机器类型命名为 RISC, 而将以前的那种称为 CISC, 因为以前没有必要给一种几乎是通用的指令集格式起名字。

比较 CISC 和最初的 RISC 指令集, 我们发现下面这些一般特性。

CISC	早期的 RISC
指令数量很多。Intel 描述全套指令的文档[51]有 1200 多页。	指令数量少得多。通常少于 100 个。
有些指令的延迟很长。包括将一个整块从内存的一个部分复制到另一部分的指令, 以及其他一些将多个寄存器的值复制到内存或从内存复制到多个寄存器的指令。	没有较长延迟的指令。有些早期的 RISC 机器甚至没有整数乘法指令, 要求编译器通过一系列加法来实现乘法。
编码是可变长度的。x86-64 的指令长度可以是 1~15 个字节。	编码是固定长度的。通常所有的指令都编码为 4 个字节。
指定操作数的方式很多样。在 x86-64 中, 内存操作数指示符可以有许多不同的组合, 这些组合由偏移量、基址和变址寄存器以及伸缩因子组成。	简单寻址方式。通常只有基址和偏移量寻址。
可以对内存和寄存器操作数进行算术和逻辑运算。	只能对寄存器操作数进行算术和逻辑运算。允许使用内存引用的只有 load 和 store 指令, load 是从内存读到寄存器, store 是从寄存器写到内存。这种方法被称为 load/store 体系结构。
对机器级程序来说实现细节是不可见的。ISA 提供了程序和如何执行程序之间的清晰的抽象。	对机器级程序来说实现细节是可见的。有些 RISC 机器禁止某些特殊的指令序列, 而有些跳转要到下一条指令执行完了以后才会生效。编译器必须在这些约束条件下进行性能优化。
有条件码。作为指令执行的副产品, 设置了一些特殊的标志位, 可以用于条件分支检测。	没有条件码。相反, 对条件检测来说, 要用明确的测试指令, 这些指令会将测试结果放在一个普通的寄存器中。
栈密集的过程链接。栈被用来存取过程参数和返回地址。	寄存器密集的过程链接。寄存器被用来存取过程参数和返回地址。因此有些过程能完全避免内存引用。通常处理器有更多的(最多的有 32 个)寄存器。

Y86-64 指令集既有 CISC 指令集的属性, 也有 RISC 指令集的属性。和 CISC 一样, 它有条件码、长度可变的指令, 并用栈来保存返回地址。和 RISC 一样的是, 它采用 load/store 体系结构和规则编码, 通过寄存器来传递过程参数。Y86-64 指令集可以看成是采用 CISC 指令集(x86), 但又根据某些 RISC 的原理进行了简化。

旁注 RISC 与 CISC 之争

20 世纪 80 年代, 计算机体系结构领域里关于 RISC 指令集和 CISC 指令集优缺点的争论十分激烈。RISC 的支持者声称在给定硬件数量的情况下, 通过结合简约式指令集设计、高级编译器技术和流水线化的处理器实现, 他们能够得到更强的计算能力。而 CISC 的拥趸反驳说要完成一个给定的任务只需要用较少的 CISC 指令, 所以他们的机器能够获得更高的总体性能。

大多数公司都推出了 RISC 处理器系列产品, 包括 Sun Microsystems(SPARC)、IBM 和 Motorola(PowerPC), 以及 Digital Equipment Corporation(Alpha)。一家英国公司 Acorn Computers Ltd. 提出了自己的体系结构——ARM(最开始是“Acorn RISC Machine”的首字母缩写), 广泛应用在嵌入式系统中(比如手机)。

20 世纪 90 年代早期, 争论逐渐平息, 因为事实已经很清楚了, 无论是单纯的 RISC 还是单纯的 CISC 都不如结合两者思想精华的设计。RISC 机器发展进化的过程中, 引入了更多的指令, 而许多这样的指令都需要执行多个周期。今天的 RISC 机器的指令表中有几百条指令, 几乎与“精简指令集机器”的名称不相符了。那种将实现细节暴露给机器级程序的思想已经被证明是目光短浅的。随着使用更加高级硬件结构的新处理器模型的开发, 许多实现细节已经变得很落后了, 但它们仍然是指令集的一部分。不过, 作为 RISC 设计的核心的指令集仍然是非常适合在流水线化的机器上执行的。

比较新的 CISC 机器也利用了高性能流水线结构。就像我们将在 5.7 节中讨论的那样, 它们读取 CISC 指令, 并动态地翻译成比较简单的、像 RISC 那样的操作的序列。例如, 一条将寄存器和内存相加的指令被翻译成三个操作: 一个是读原始的内存值, 一个是执行加法运算, 第三就是将和写回内存。由于动态翻译通常可以在实际指令执行前进行, 处理器仍然可以保持很高的执行速率。

除了技术因素以外, 市场因素也在决定不同指令集是否成功中起了很重要的作用。通过保持与现有处理器的兼容性, Intel 以及 x86 使得从一代处理器迁移到下一代变得很容易。由于集成电路技术的进步, Intel 和其他 x86 处理器制造商能够克服原来 8086 指令集设计造成的低效率, 使用 RISC 技术产生出与最好的 RISC 机器相当的性能。正如我们在第 3.1 节中看到的那样, IA32 发展演变到 x86-64 提供了一个机会, 使得能够将 RISC 的一些特性结合到 x86 中。在桌面、便携计算机和基于服务器的计算领域里, x86 已经占据了完全的统治地位。

RISC 处理器在嵌入式处理器市场上表现得非常出色, 嵌入式处理器负责控制移动电话、汽车刹车以及因特网电器等系统。在这些应用中, 降低成本和功耗比保持后向兼容性更重要。就出售的处理器数量来说, 这是个非常广阔而迅速成长着的市场。

4.1.4 Y86-64 异常

对 Y86-64 来说, 程序员可见的状态(图 4-1)包括状态码 Stat, 它描述程序执行的总体状态。这个代码可能的值如图 4-5 所示。代码值 1, 命名为 AOK, 表示程序执行正常,

而其他一些代码则表示发生了某种类型的异常。代码 2，命名为 HLT，表示处理器执行了一条 halt 指令。代码 3，命名为 ADR，表示处理器试图从一个非法内存地址读或者向一个非法内存地址写，可能是当取指令的时候，也可能是当读或者写数据的时候。我们会限制最大的地址(确切的限定值因实现而异)，任何访问超出这个限定值的地址都会引发 ADR 异常。代码 4，命名为 INS，表示遇到了非法的指令代码。

值	名字	含义
1	AOK	正常操作
2	HLT	遇到器执行 halt 指令
3	ADR	遇到非法地址
4	INS	遇到非法指令

对于 Y86-64，当遇到这些异常的时候，我们就简单地让处理器停止执行指令。在更完整的设计中，处理器通常会调用一个异常处理程序

图 4-5 Y86-64 状态码。在我们的设计中，任何 AOK 以外的代码都会使处理器停止

(exception handler)，这个过程被指定用来处理遇到的某种类型的异常。就像在第 8 章中讲述的，异常处理程序可以被配置成不同的结果，例如，中止程序或者调用一个用户自定义的信号处理程序(signal handler)。

4.1.5 Y86-64 程序

图 4-6 给出了下面这个 C 函数的 x86-64 和 Y86-64 汇编代码：

<pre>1 long sum(long *start, long count) 2 { 3 long sum = 0; 4 while (count) { 5 sum += *start; 6 start++; 7 count--; 8 } 9 return sum; 10 }</pre>	<pre>long sum(long *start, long count) start in %rdi, count in %rsi 1 sum: 2 movl \$0, %eax sum = 0 3 jmp .L2 Goto test 4 .L3: loop: 5 addq (%rdi), %rax Add *start to sum 6 addq \$8, %rdi start++ 7 subq \$1, %rsi count-- 8 .L2: test: 9 testq %rsi, %rsi Test sum 10 jne .L3 If !=0, goto loop 11 rep; ret Return</pre>	<pre>long sum(long *start, long count) start in %rdi, count in %rsi 1 sum: 2 irmovq \$8,%r8 Constant 8 3 irmovq \$1,%r9 Constant 1 4 xorq %rax,%rax sum = 0 5 andq %rsi,%rsi Set CC 6 jmp test Goto test 7 loop: 8 mrmovq (%rdi),%r10 Get *start 9 addq %r10,%rax Add to sum 10 addq %r8,%rdi start++ 11 subq %r9,%rsi count--, Set CC 12 test: 13 jne loop Stop when 0 14 ret Return</pre>
---	---	--

图 4-6 Y86-64 汇编程序与 x86-64 汇编程序比较。Sum 函数计算一个整数数组的和。Y86-64 代码与 x86-64 代码遵循了相同的通用模式

x86-64 代码是由 GCC 编译器产生的。Y86-64 代码与之类似，但有以下不同点：

- Y86-64 将常数加载到寄存器(第 2~3 行)，因为它在算术指令中不能使用立即数。
- 要实现从内存读取一个数值并将其与一个寄存器相加，Y86-64 代码需要两条指令(第 8~9 行)，而 x86-64 只需要一条 `addq` 指令(第 5 行)。
- 我们手工编写的 Y86-64 实现有一个优势，即 `subq` 指令(第 11 行)同时还设置了条件码，因此 GCC 生成代码中的 `testq` 指令(第 9 行)就不是必需的。不过为此，Y86-64 代码必须用 `andq` 指令(第 5 行)在进入循环之前设置条件码。

图 4-7 给出了用 Y86-64 汇编代码编写的一个完整的程序文件的例子。这个程序既包括数据，也包括指令。伪指令(directive)指明应该将代码或数据放在什么位置，以及如何对齐。这个程序详细说明了栈的放置、数据初始化、程序初始化和程序结束等问题。

```

1  # Execution begins at address 0
2      .pos 0
3      irmovq stack, %rsp      # Set up stack pointer
4      call main               # Execute main program
5      halt                   # Terminate program
6
7  # Array of 4 elements
8      .align 8
9  array:
10     .quad 0x000d000d000d
11     .quad 0x00c000c000c0
12     .quad 0x0b000b000b00
13     .quad 0xa000a000a000
14
15  main:
16     irmovq array,%rdi
17     irmovq $4,%rsi
18     call sum                 # sum(array, 4)
19     ret
20
21  # long sum(long *start, long count)
22  # start in %rdi, count in %rsi
23  sum:
24     irmovq $8,%r8           # Constant 8
25     irmovq $1,%r9           # Constant 1
26     xorq %rax,%rax          # sum = 0
27     andq %rsi,%rsi          # Set CC
28     jmp test                # Goto test
29  loop:
30     mrmovq (%rdi),%r10       # Get *start
31     addq %r10,%rax           # Add to sum
32     addq %r8,%rdi            # start++
33     subq %r9,%rsi            # count--. Set CC
34  test:
35     jne loop                 # Stop when 0
36     ret                      # Return
37
38  # Stack starts here and grows to lower addresses
39     .pos 0x200
40  stack:

```

图 4-7 用 Y86-64 汇编代码编写的一个例子程序。调用 `sum` 函数来计算一个具有 4 个元素的数组的和

在这个程序中,以“.”开头的词是汇编器伪指令(assembly directives),它们告诉汇编器调整地址,以便在那儿产生代码或插入一些数据。伪指令`.pos 0`(第2行)告诉汇编器应该从地址0处开始产生代码。这个地址是所有Y86-64程序的起点。接下来的一条指令(第3行)初始化栈指针。我们可以看到程序结尾处(第40行)声明了标号`stack`,并且用一个`.pos`伪指令(第39行)指明地址`0x200`。因此栈会从这个地址开始,向低地址增长。我们必须保证栈不会增长得太大以至于覆盖了代码或者其他程序数据。

程序的第8~13行声明了一个4个字的数组,值分别为

```
0x000d000d000d000d, 0x00c000c000c000c0
0x0b000b000b000b00, 0xa000a000a000a000
```

标号`array`表明了这个数组的起始,并且在8字节边界处对齐(用`.align`伪指令指定)。第16~19行给出了“main”过程,在过程中对那个四字数组调用了`sum`函数,然后停止。

正如例子所示,由于我们创建Y86-64代码的唯一工具是汇编器,程序员必须执行本来通常交给编译器、链接器和运行时系统来完成任务。幸好我们只用Y86-64来写一些小的程序,对此一些简单的机制就足够了。

图4-8是YAS的汇编器对图4-7中代码进行汇编的结果。为了便于理解,汇编器的输出结果是ASCII码格式。汇编文件中有指令或数据的行上,目标代码包含一个地址,后面跟着1~10个字节的值。

我们实现了一个指令集模拟器,称为YIS,它的目的是模拟Y86-64机器代码程序的执行,而不用试图去模拟任何具体处理器实现的行为。这种形式的模拟有助于在有实际硬件可用之前调试程序,也有助于检查模拟硬件或者在硬件上运行程序的结果。用YIS运行例子的目标代码,产生如下输出:

```
Stopped in 34 steps at PC = 0x13. Status 'HLT', CC Z=1 S=0 O=0
```

```
Changes to registers:
```

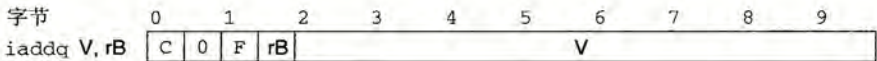
```
%rax: 0x0000000000000000      0x0000abcdabcdabcd
%rsp: 0x0000000000000000      0x0000000000000200
%rdi: 0x0000000000000000      0x0000000000000038
%r8:  0x0000000000000000      0x0000000000000008
%r9:  0x0000000000000000      0x0000000000000001
%r10: 0x0000000000000000      0x0000a000a000a000
```

```
Changes to memory:
```

```
0x01f0: 0x0000000000000000      0x0000000000000055
0x01f8: 0x0000000000000000      0x0000000000000013
```

模拟输出的第一行总结了执行以及PC和程序状态的结果值。模拟器只打印出在模拟过程中被改变了的寄存器或内存中的字。左边是原始值(这里都是0),右边是最终的值。从输出中我们可以看到,寄存器`%rax`的值为`0xabcdabcdabcdabcd`,即传给子函数`sum`的四元素数组的和。另外,我们还能看到栈从地址`0x200`开始,向下增长,栈的使用导致内存地址`0x1f0~0x1f8`发生了变化。可执行代码的最大地址为`0x090`,所以数值的入栈和出栈不会破坏可执行代码。

 **练习题 4.3** 机器级程序中常见的模式之一是将一个常数值与一个寄存器相加。利用目前已给出的Y86-64指令,实现这个操作需要一条`irmovq`指令把常数加载到寄存器,然后一条`addq`指令把这个寄存器值与目标寄存器值相加。假设我们想增加一条新指令`iaddq`,格式如下:



该指令将常数值 V 与寄存器 rB 相加。

使用 iaddq 指令重写图 4-6 的 Y86-64 sum 函数。在之前的代码中，我们用寄存器 %r8 和 %r9 来保存常数值。现在，我们完全可以避免使用这些寄存器。

```

                                | # Execution begins at address 0
0x000:                          | .pos 0
0x000: 30f40002000000000000    | irmovq stack, %rsp      # Set up stack pointer
0x00a: 80380000000000000000    | call main               # Execute main program
0x013: 00                      | halt                   # Terminate program
                                |
                                | # Array of 4 elements
0x018:                          | .align 8
0x018:                          | array:
0x018: 0d000d000d000000       | .quad 0x000d000d000d
0x020: c000c000c0000000       | .quad 0x00c000c000c0
0x028: 000b000b000b0000       | .quad 0x0b000b000b00
0x030: 00a000a000a00000       | .quad 0xa000a000a000
                                |
0x038:                          | main:
0x038: 30f71800000000000000    | irmovq array, %rdi
0x042: 30f60400000000000000    | irmovq $4, %rsi
0x04c: 80560000000000000000    | call sum                # sum(array, 4)
0x055: 90                      | ret
                                |
                                | # long sum(long *start, long count)
                                | # start in %rdi, count in %rsi
0x056:                          | sum:
0x056: 30f80800000000000000    | irmovq $8, %r8          # Constant 8
0x060: 30f90100000000000000    | irmovq $1, %r9          # Constant 1
0x06a: 6300                    | xorq %rax, %rax         # sum = 0
0x06c: 6266                    | andq %rsi, %rsi         # Set CC
0x06e: 70870000000000000000    | jmp test               # Goto test
0x077:                          | loop:
0x077: 50a70000000000000000    | mrmovq (%rdi), %r10     # Get *start
0x081: 60a0                    | addq %r10, %rax         # Add to sum
0x083: 6087                    | addq %r8, %rdi         # start++
0x085: 6196                    | subq %r9, %rsi         # count--. Set CC
0x087:                          | test:
0x087: 74770000000000000000    | jne loop               # Stop when 0
0x090: 90                      | ret                    # Return
                                |
                                | # Stack starts here and grows to lower addresses
0x200:                          | .pos 0x200
0x200:                          | stack:
```

图 4-8 YAS 汇编器的输出。每一行包含一个十六进制的地址，以及字节数在 1~10 之间的目标代码

 **练习题 4.4** 根据下面的 C 代码，用 Y86-64 代码来实现一个递归求和函数 rsum：

```
long rsum(long *start, long count)
{
    if (count <= 0)
        return 0;
    return *start + rsum(start+1, count-1);
}
```

使用与 x86-64 代码相同的参数传递和寄存器保存方法。在一台 x86-64 机器上编译这段 C 代码，然后再把那些指令翻译成 Y86-64 的指令，这样做可能会很有帮助。

 **练习题 4.5** 修改 sum 函数的 Y86-64 代码(图 4-6)，实现函数 absSum，它计算一个数组的绝对值的和。在内循环中使用条件跳转指令。

 **练习题 4.6** 修改 sum 函数的 Y86-64 代码(图 4-6)，实现函数 absSum，它计算一个数组的绝对值的和。在内循环中使用条件传送指令。

4.1.6 一些 Y86-64 指令的详情

大多数 Y86-64 指令是以一种直接明了的方式修改程序状态的，所以定义每条指令想要达到的结果并不困难。不过，两个特别的指令的组合需要特别注意一下。

pushq 指令会把栈指针减 8，并且将一个寄存器值写入内存中。因此，当执行 pushq %rsp 指令时，处理器的行为是不确定的，因为要入栈的寄存器会被同一条指令修改。通常有两种不同的约定：1) 压入 %rsp 的原始值，2) 压入减去 8 的 %rsp 的值。

对于 Y86-64 处理器来说，我们采用和 x86-64 一样的做法，就像下面这个练习题确定出的那样。

 **练习题 4.7** 确定 x86-64 处理器上指令 pushq %rsp 的行为。我们可以通过阅读 Intel 关于这条指令的文档来了解它们的做法，但更简单的方法是在实际的机器上做个实验。C 编译器正常情况下是不会产生这条指令的，所以我们必须用手工生成的汇编代码来完成这一任务。下面是我们写的一个测试程序(网络旁注 ASM:EASM，描绘如何编写 C 代码和手写汇编代码结合的程序)：

```
1  .text
2  .globl pushtest
3  pushtest:
4      movq    %rsp, %rax    Copy stack pointer
5      pushq   %rsp          Push stack pointer
6      popq    %rdx          Pop it back
7      subq    %rdx, %rax    Return 0 or 8
8      ret
```

在实验中，我们发现函数 pushtest 总是返回 0，这表示在 x86-64 中 pushq %rsp 指令的行为是怎样的呢？

对 popq %rsp 指令也有类似的歧义。可以将 %rsp 置为从内存中读出的值，也可以置为加了增量后的栈指针。同练习题 4.7 一样，让我们做个实验来确定 x86-64 机器是怎么处理这条指令的，然后 Y86-64 机器就采用同样的方法。

 **练习题 4.8** 下面这个汇编函数让我们确定 x86-64 上指令 popq %rsp 的行为：

```
1  .text
2  .globl poptest
3  poptest:
```



```

4      movq    %rsp, %rdi    Save stack pointer
5      pushq   $0xabcd      Push test value
6      popq    %rsp          Pop to stack pointer
7      movq    %rsp, %rax    Set popped value as return value
8      movq    %rdi, %rsp    Restore stack pointer
9      ret

```

我们发现函数总是返回 0xabcd。这表示 popq %rsp 的行为是怎样的？还有什么其他 Y86-64 指令也会有相同的行为吗？

旁注 正确了解细节：x86 模型间的不一致

练习题 4.7 和练习题 4.8 可以帮助我们确定对于压入和弹出栈指针指令的一致惯例。看上去似乎没有理由会执行这样两种操作，那么一个很自然的问题就是“为什么要担心这样一些吹毛求疵的细节呢？”

从下面 Intel 关于 PUSH 指令的文档[51]的节选中，可以学到关于这个一致的重要性的有用的教训：

对于 IA-32 处理器，从 Intel 286 开始，PUSH ESP 指令将 ESP 寄存器的值压入栈中，就好像它存在于这条指令被执行之前。（对于 Intel 64 体系结构、IA-32 体系结构的实地址模式和虚 8086 模式来说也是这样。）对于 Intel® 8086 处理器，PUSH SP 将 SP 寄存器的新值压入栈中（也就是减去 2 之后的值）。（PUSH ESP 指令。Intel 公司。50。）

虽然这个说明的具体细节可能难以理解，但是我们可以看到这条注释说明的是当执行压入栈指针寄存器指令时，不同型号的 x86 处理器会做不同的事情。有些会压入原始的值，而有些会压入减去后的值。（有趣的是，对于弹出栈指针寄存器没有类似的歧义。）这种不一致有两个缺点：

- 它降低了代码的可移植性。取决于处理器模型，程序可能会有不同的行为。虽然这样特殊的指令并不常见，但是即使是潜在的不兼容也可能带来严重的后果。
- 它增加了文档的复杂性。正如在这里我们看到的那样，需要一个特别的说明来澄清这些不同之处。即使没有这样的特殊情况，x86 文档就已经够复杂的了。

因此我们的结论是，从长远来看，提前了解细节，力争保持完全的一致能够节省很多的麻烦。

4.2 逻辑设计和硬件控制语言 HCL

在硬件设计中，用电子电路来计算对位进行运算的函数，以及在各种存储器单元中存储位。大多数现代电路技术都是用信号线上的高电压或低电压来表示不同的位值。在当前的技术中，逻辑 1 是用 1.0 伏特左右的高电压表示的，而逻辑 0 是用 0.0 伏特左右的低电压表示的。要实现一个数字系统需要三个主要的组成部分：计算对位进行操作的函数的组合逻辑、存储位的存储器单元，以及控制存储器单元更新的时钟信号。

本节简要描述这些不同的组成部分。我们还将介绍 HCL (Hardware Control Language, 硬件控制语言)，用这种语言来描述不同处理器设计的控制逻辑。在此我们只是简略地描述 HCL，HCL 完整的参考请见网络旁注 ARCH:HCL。

旁注 现代逻辑设计

曾经，硬件设计者通过描绘示意性的逻辑电路图来进行电路设计（最早是用纸和笔，后来是用计算机图形终端）。现在，大多数设计都是用硬件描述语言 (Hardware Description